

M.Abdullah.Mahmood

BCS-3B

22F-3289

Data Structures Lab -3

To: Sir Abdul Qadeer

Task 1 :

Code:

```
#include<iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node():next(nullptr) {};
    Node(int d) : data(d), next(nullptr) {};
};

class LinkedList {
public:
    Node* head;
    LinkedList():head(nullptr) {};

    void insert(int val) {
        Node* temp = new Node(val);
        if (head == NULL)
            head = temp;
        else {
            Node* cur = head;
            temp->next = cur;
            head = temp;
        }
        return;
    }

    void search(int val) {
        Node* cur = head;
        int pos = 0;
        while (cur) {
            ++pos;
            if (val == cur->data) {
                cout << "its found at " << pos << " index" << endl;
            }
            cur = cur->next;
        }
    }
};
```

```

        return;
    }
    else {
        cur = cur->next;
    }
}

void update(int val,int updateVal = 9) {
    Node* cur = head;
    while (cur) {
        if (val == cur->data) {
            cur->data = updateVal;
            return;
        }
        else {
            cur = cur->next;
        }
    }
    cout << "not found \n";
}

void reverse() {
    if (head == NULL) {
        cout << "list is empty \n";
        return;
    }

    Node* cur = head;
    Node* prev =NULL , *next ;

    while (cur) {
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    head = prev;

    return;
}

void secondMax() {
    int max1 = -1, max2 ;
    Node* cur = head;
    while (cur) {
        if (cur->data > max1) {
            max2 = max1;
            max1 = cur->data;
        }
        else if (cur->data < max1 && cur->data>max2) {
            max2 = cur->data;
        }
        cur = cur->next;
    }
    cout << "second max is : " << max2 << endl;
}

```

```

        void display() {
            Node* cur = head;
            while (cur) {
                cout << cur->data << " " << endl;
                cur = cur->next;
            }
            cout << endl;
            return;
        }

~LinkedList() {}
};

int main()
{
    LinkedList L;

    L.insert(1);
    L.insert(2);
    L.insert(3);
    L.insert(4);

    L.display();

    L.search(2);

    L.update(2);
    L.display();

    L.reverse();
    L.display();

    L.secondMax();

    system("pause");
    return 0;
}

```

Output:

```
C:\Users\F223289\Desktop\la X + v
4
3
2
1

its found at 3 index
4
3
9
1

1
9
3
4

second max is : 4
Press any key to continue . . .
```

Task 3:

```
#include<iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node():next(nullptr) {};
    Node(int d) : data(d), next(nullptr) {};
};

class LinkedList {
public:
    Node* head;
    LinkedList() :head(nullptr) {};

    void insert(int val) {
        Node* temp = new Node(val);
```

```

        if (head == NULL)
            head = temp;
        else {
            Node* cur = head;
            temp->next = cur;
            head = temp;
        }
        return;
    }

    Node* reverse() {

        Node* cur = head;
        Node* prev = NULL , *next ;

        while (cur) {
            next = cur->next;
            cur->next = prev;
            prev = cur;
            cur = next;
        }

        return prev;
    }

    void display() {
        Node* cur = head;
        while (cur) {
            cout << cur->data << " " << endl;
            cur = cur->next;
        }
        cout << endl;
    }
};

void mergeLists(LinkedList& train1, LinkedList& train2) {

    train1.display();
    cout << endl;

    while (train1.head->next) {
        train1.head = train1.head->next;
    }

    train1.display();
    cout << endl;

    train2.display();
    cout << endl;
    train1.head->next=train2.reverse();

    train1.display();
    cout << endl;
}

int main()
{
    LinkedList t1, t2;

```

```

        t1.insert(1);
        t1.insert(2);
        t1.insert(3);
        t1.insert(4);

        t2.insert(5);
        t2.insert(6);
        t2.insert(7);
        t2.insert(8);

        t1.display();
        t2.display();

        mergeLists(t1, t2);

        system("pause");
    return 0;
}

```

Task 2:

```

#include<iostream>
using namespace std;

class Node {
public:
    string name;
    string roll;
    string course;
    float gpa;

    Node* next;

    Node() :next(nullptr) {};
    Node(string n, string r, string c1, float g) {
        name = n;
        roll = r;
        course[1] = c1;
        gpa = g;
    }
};

class LinkedList {
public:
    Node* head;

    LinkedList() : head(nullptr) {};

    void insertData() {
        Node* temp = head;
    }
}

```

```

float gpa;
string name, roll , course;
cout << "enter gpa : ";
cin >> gpa;
if (gpa < 3.0) {
    cout << "cant enter \n";
    return;
}
else {
    cin >> gpa;
    cin >> name;
    cin >> roll;
    for (int i = 0; i < 3; ++i) {
        cin >> course[i];
    }
    if (head == NULL) {
        Node* temp1 = new Node(name, roll, course,gpa);
        head = temp1;
    }
    else {
        Node* temp1 = new Node(name, roll, course, gpa);
        while (temp) {
            temp = temp->next;
        }
        temp->next = temp1;
    }
}

}

void show() {
    Node* temp = head;
    while (temp) {
        cout << temp->name << temp->gpa << temp->roll << temp->course;
    }
}

};

```